TOBB UNIVERSITY OF ECONOMICS AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

BİL 481 – SOFTWARE ENGINEERING PROJECT ASSIGNMENT 2: DESIGN DOCUMENT

PROJECT NAME: SWAPLY

AUTHORS: Arda Burak Zararsız

Yusuf Çakır

Yusuf Enes Kütük

Eren Öz

Furkan Yılmaz

DATE: 05.03.2026

Table of Contents

1. Task Matrix 2
2. System Overview 3
3. Implementation Details 3
4. Use Case Support in Design 4
5. Design Decisions 5

System Overview & Architecture: Eren Öz, Yusuf Enes Kütük

Implementation Details & Codebase Structure: Yusuf Çakır, Yusuf Enes Kütük

Use Case Support & Diagrams: Arda Burak Zararsız, Furkan Yılmaz

Design Decisions & Justifications: Eren Öz, Yusuf Çakır

Document Formatting & Final Review: Eren Öz, Arda Burak Zararsız

System Overview

Brief Project Description: Swaply, üniversite öğrencilerinin sahip oldukları bilgi ve yetenekleri birbirleriyle takas etmelerini sağlayan konum tabanlı bir mobil platformdur. Projemiz, öğrencilerin kampüs içindeki "akran öğrenimi" potansiyelini kullanarak bilgiye erişimini kolaylaştırmayı amaçlamaktadır.

System Architecture: We are utilizing a client-server model based on a serverless architecture. The client side is a mobile application, while the backend is entirely managed through managed cloud services. This layered approach separates the UI rendering from database operations and authentication logic.

Technology Stack:

- Frontend: React Native, Expo.
- Backend & Database: Firebase Authentication, Cloud Firestore (NoSQL).
- Push Notifications: Firebase Cloud Messaging (FCM).
- Design Tool: Figma.

Implementation Details

Codebase Structure: The React Native project is organized into modular folders to maintain separation of concerns:

- /src/assets: Images, fonts, and icons.
- /src/components: Reusable UI components (e.g., CustomButton, MatchCard).
- /src/screens: Main application views (e.g., LoginScreen, MapScreen, ChatScreen).
- /src/navigation: React Navigation setup (Stack and Tab navigators).
- /src/services: Firebase API wrappers and configuration.
- /src/utils: Helper functions (e.g., distance calculation, date formatting).

Key Implementations: The core business logic resides in the matching algorithm. When a user updates their "skillsToTeach" or "skillsToLearn" array, a Firestore query checks for overlapping arrays in other user documents. If an overlap is found, a match document is created.

Component Interfaces: Key backend helper functions include:

- registerUser(email, password): Validates @edu.tr format and creates Auth record.
- findMatches(uid): Queries Firestore for complementary skill arrays.
- sendMessage(chatId, senderId, text): Appends a new message object to the chat subcollection.

Visual Interfaces: The following screens were designed in Figma and represent the core user flows of the Swaply application. The wireframes cover the full journey from onboarding to skill matching, real-time chat, and profile management.

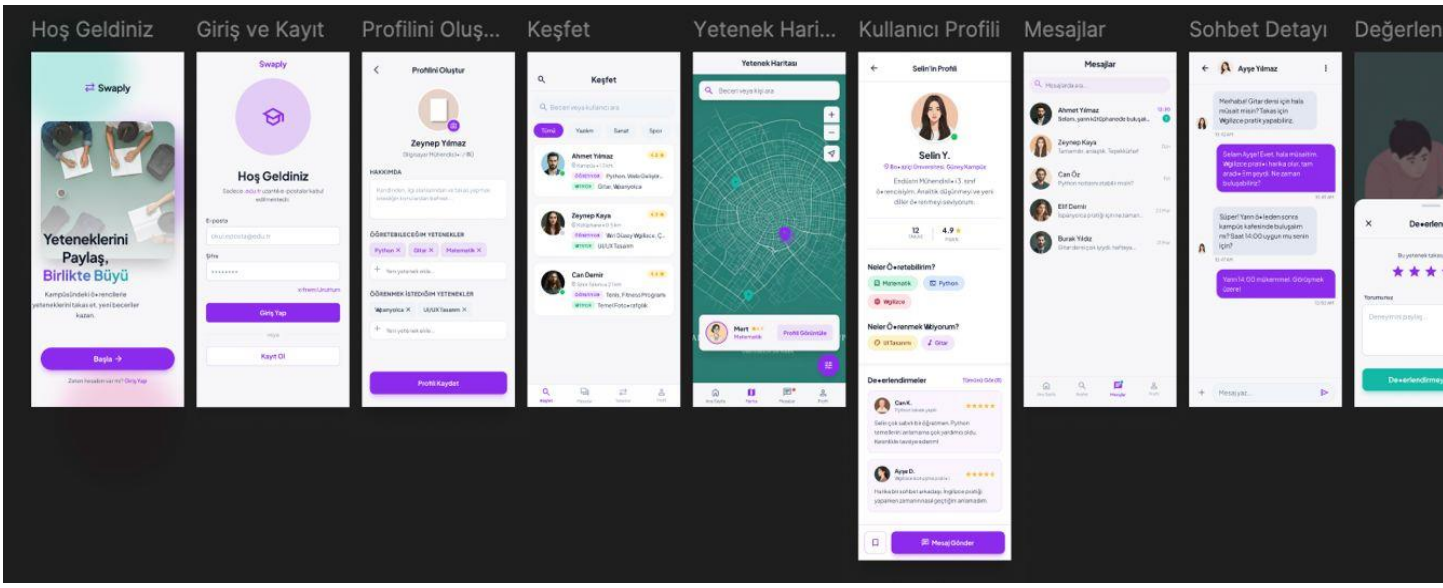

Figure 1: Swaply UI Screens — Onboarding, Login, Profile Setup, Discover, Skill Map, User Profile, Messages, Chat, Rating, and Settings

Use Case Support in Design

Use Case Selection & Requirement Mapping: We have selected the following 4 core use cases from our Requirements Document to detail our design:

1. User Registration & Verification -> Maps to FR1
2. Skill-Based Match Notification -> Maps to FR3
3. Real-Time Chat -> Maps to FR4
4. Location-Based Matching (Match-Map) -> Maps to FR6

Use Case Design & Data Flow:

Use Case 1: User Registration Data Flow: User enters credentials -> Client validates @edu.tr regex -> Client calls Firebase Auth SDK -> Firebase creates account and sends verification email -> User document created in Firestore upon verification.

Use Case 2: Skill-Based Match Notification Data Flow: User A adds a skill -> Client updates Firestore user document -> Client runs findMatches() query -> If match found, Match document is created -> FCM sends push notification payload to both User A and User B device tokens.

Use Case 3: Real-Time Chat Data Flow: Users open chat -> Client attaches onSnapshot listener to the specific chat's "messages" subcollection -> User sends message -> Client writes to Firestore -> Listener immediately triggers UI update for both users.

Use Case 4: Location-Based Matching Data Flow: User grants GPS permission -> Expo Location SDK fetches coordinates -> Client queries Firestore for users -> Client filters results using a bounding box calculation for a 5 km radius -> React Native Maps renders coordinate data as pins on the UI.

Design Decisions

Technology Comparisons:

Comparison 1: Firebase vs. Custom Node.js/Express Backend

We evaluated building a custom REST API using Node.js and MongoDB versus using Firebase. A custom Node.js backend offers more flexibility for complex geographical queries (using MongoDB GeoJSON) and fine-grained control over server-side logic. However, it requires significant setup, deployment infrastructure, and ongoing maintenance. Firebase provides real-time data syncing out-of-the-box (critical for our Chat feature), built-in Authentication, and a generous free tier (Spark Plan). Given our 10-week timeline and zero-budget constraint, Firebase was the clear choice.

Comparison 2: React Native (Expo) vs. Flutter

For the cross-platform mobile framework, we compared React Native (via Expo) against Flutter. Flutter offers a highly performant rendering engine and a rich widget library, making it excellent for pixel-perfect UIs. However, Flutter uses Dart, a language our team has no prior experience with, introducing a steep learning curve. React Native uses JavaScript/TypeScript, which all team members are proficient in. Additionally, the Expo ecosystem simplifies device testing (via Expo Go) and provides ready-to-use libraries for GPS, notifications, and maps—all features required by Swaply. For these reasons, React Native with Expo was selected as our frontend framework.

Decision Justifications: We chose Firebase because our team has strict time constraints (10 weeks). Firebase's "Spark Plan" perfectly covers our budget constraint (0 TL) while providing all necessary services (Auth, Firestore, FCM) without the overhead of server maintenance. We decided to handle the location distance calculation on the client side to bypass Firestore's native geo-query limitations. 5. GitHub Commit Requirement

Code Implementations & Interfaces: All implementations for the 4 core use cases, component interfaces, and React Native UI files have been pushed to the main repository. Technology Comparisons: Code snippets demonstrating the difference between a standard REST API fetch and a Firebase onSnapshot listener have been committed to the repository in the /docs folder for comparison.